



One calm home for everything you're studying.

A personalized student productivity & academic management workspace.

HTML5

•

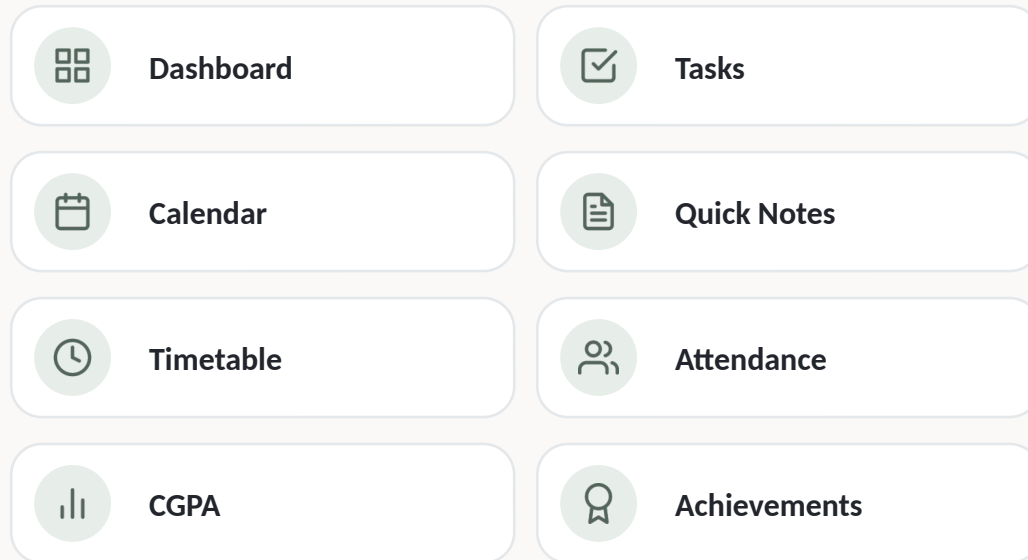
CSS3

•

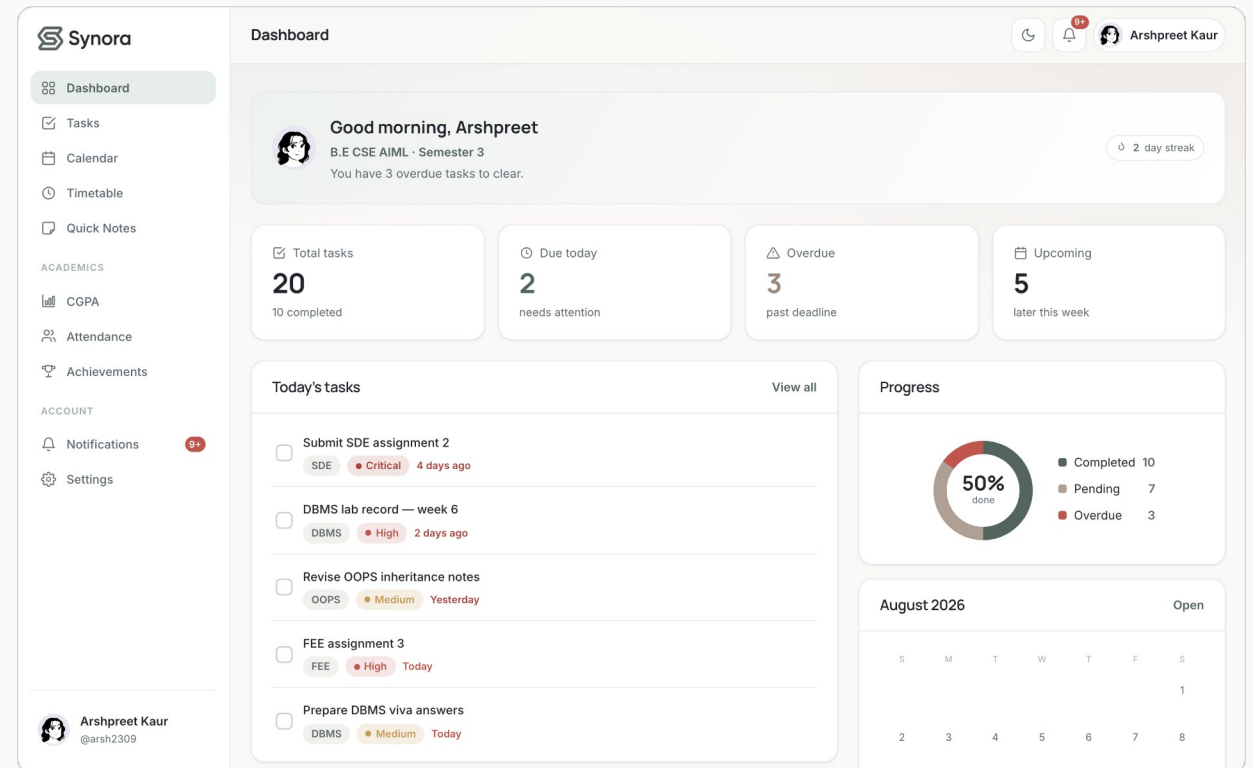
VANILLA JAVASCRIPT

What is Synora?

Synora brings a student's entire academic workflow into one calm workspace — instead of juggling six notebooks, three apps and a group chat, everything a student actually tracks lives in a single dashboard.



One dashboard that answers “*what do I need to do today?*” at a glance.



The Synora dashboard — the demo workspace, signed in as Arshpreet Kaur.

The student workflow is scattered



Deadlines

scattered across chats and sticky notes



Notes

spread over three different apps



Timetable

a photo of the board, somewhere



Attendance

recalculated by hand every week



CGPA

a spreadsheet you forget to open

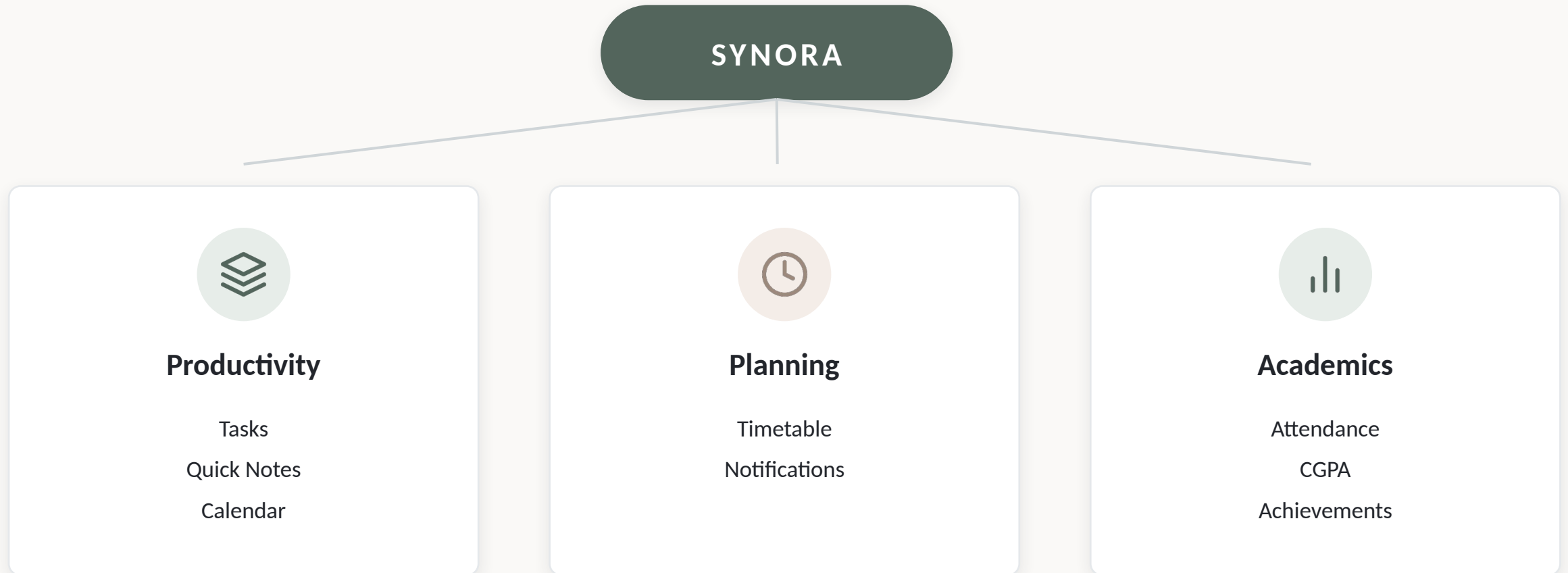


Tasks

remembered a little too late

The result Students spend more time managing their academic information than actually focusing on it.

One workspace. Every academic workflow.



Synora unifies productivity and academic tracking into a single, personalized interface.

Why Synora?



Unified

One workspace instead of six disconnected tools.



Personalized

Every user configures their own subjects & workspace.



Student-centric

Designed around real student workflows, not generic to-dos.



Lightweight

Core web tech — no framework, no build step, no bundler.



Responsive

Works from a 320px phone to a large monitor.



Extensible

Architecture ready to grow into a cloud platform.

Everything a student needs



Productivity

Tasks (full CRUD)

Calendar

Quick Notes



Academics

Attendance

CGPA calculator

Timetable



Motivation

Achievements

Progress ring

Completion streak



Personalization

Profile & avatars

Light / dark theme

Subjects & credits



Communication

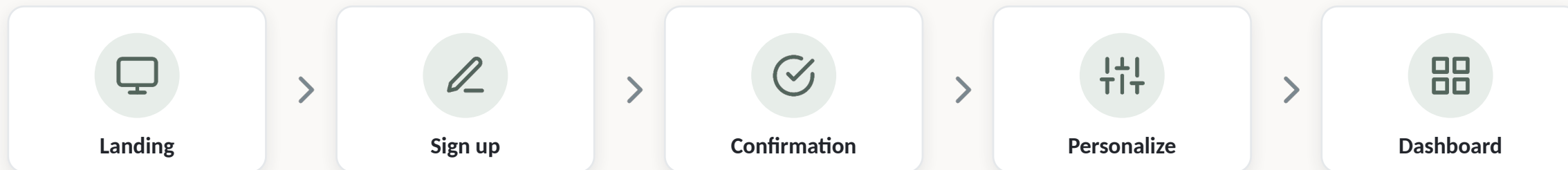
Notifications

Due / overdue alerts

Attendance warnings

Twelve features, one visual language — every screen shares the same tokens, components and dashboard.

From sign up to study



PERSONALIZE SYNORA — 6 STEPS

- 1 Avatar
- 2 Program & Semester
- 3 Subjects & Credits
- 4 Attendance target
- 5 Appearance
- 6 Review & finish



Daily workspace

Tasks • Calendar • Quick Notes • Timetable

Everything you touch every day, in one place.

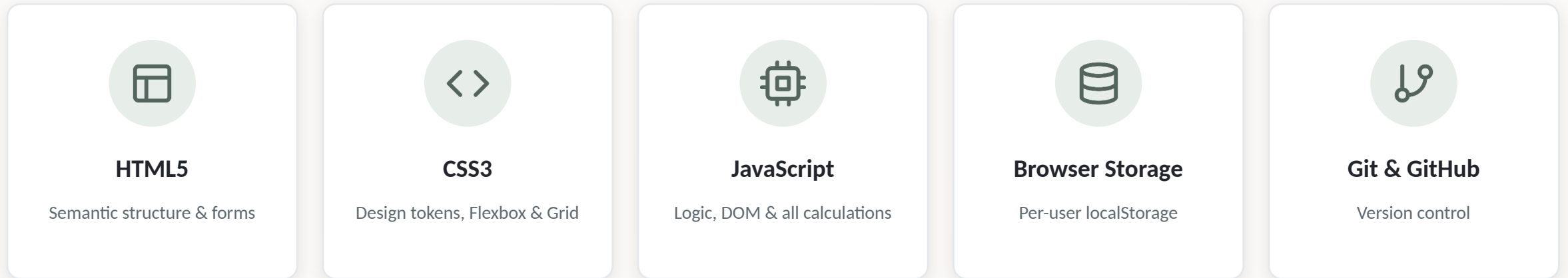


Academic tracking

Attendance • CGPA • Achievements • Progress

The numbers that decide your semester, kept current.

Built on core web technologies



No framework. No build step. No bundler. The only external resource is the Google Fonts stylesheet (Inter + Manrope) — it degrades to system fonts without it.

DEVELOPMENT WORKFLOW



Written in VS Code • debugged and responsive-tested with Chrome DevTools • versioned with Git & hosted on GitHub.

From the syllabus to a real product

HTML5 & CSS3	In Synora
Semantic HTML	header / nav / main / section / footer
Forms & inputs	sign-up, sign-in, contact, task forms
Accessibility	labels, real buttons, focus, aria-live
Box model	cards, fields, steppers, tiles
Flexbox	navbar, toolbars, card headers
CSS Grid	dashboard, timetable, feature grids
Media queries	sidebar → drawer, 7-col → stacked
Mobile-first	~320px up to large monitors

JavaScript & Tooling	In Synora
Variables & functions	app state, reusable feature logic
Arrays & objects	subjects, tasks, users, records
Loops	dynamic rendering (map / forEach)
DOM manipulation	sidebar, charts, live lists
Events	clicks, forms, event delegation
JSON	JSON.stringify / JSON.parse
Browser storage	per-user namespaced localStorage
Git & GitHub	commits & version history

Every row is demonstrated by working code in this project — the following slides show the actual snippets.

How Synora is structured



HTML pages

index · login · signup · onboarding · dashboard · 9 feature pages



CSS

tokens · base · components · app · landing · auth · onboarding



JavaScript

theme · storage · avatars · app + 11 feature scripts



Browser storage

synora:<user>:<collection> — per-user localStorage

ARCHITECTURE IN ONE LINE

HTML → CSS tokens → storage.js → app.js → per-page script

One navigation model. app.js builds the sidebar and topbar once from a single NAV list, then feature scripts wait for a **synora:ready** event before rendering into their page.

Subjects are the source of truth. Attendance, CGPA and the timetable all reference a subject by **id** — rename or delete once, and every module follows.

Structure & accessibility

```
index.html

1 <!-- Landing contact form -->
2 <form class="contact-form" novalidate>
3   <div class="field">
4     <label for="contact-email">Email</label>
5     <input type="email" id="contact-email"
6       name="email" autocomplete="email"
7       placeholder="you@example.com" required>
8     <p class="field__error"></p>
9   </div>
10  <button type="submit"
11    class="btn btn--primary">Send</button>
12 </form>
```

SEMANTIC STRUCTURE

<header>

<nav>

<main>

<section>

<footer>

<form>

<label>

<button>

CONCEPT

Semantic HTML5 + accessible forms.

USED IN

Sign-up, sign-in, personalization, subjects & the contact form.

WHY IT MATTERS

Real <label>s, <button>s, focus states and aria-live make the app usable by keyboard and screen reader.

From structure to interface

```
tokens.css + app.css

1 :root {
2   --color-brand: #53655C; /* sage */
3   --color-accent: #B09F95; /* taupe */
4   --space-24: 24px;
5   --radius-lg: 16px;
6 }
7 .dash-cols { display: grid; gap: var(--space-24); }
8
9 @media (min-width: 1000px) {
10   .dash-cols {
11     grid-template-columns: minmax(0,1.6fr) minmax(0,1fr);
12   }
13 }
```

THE BOX MODEL



USED THROUGHOUT SYNORA

Custom properties

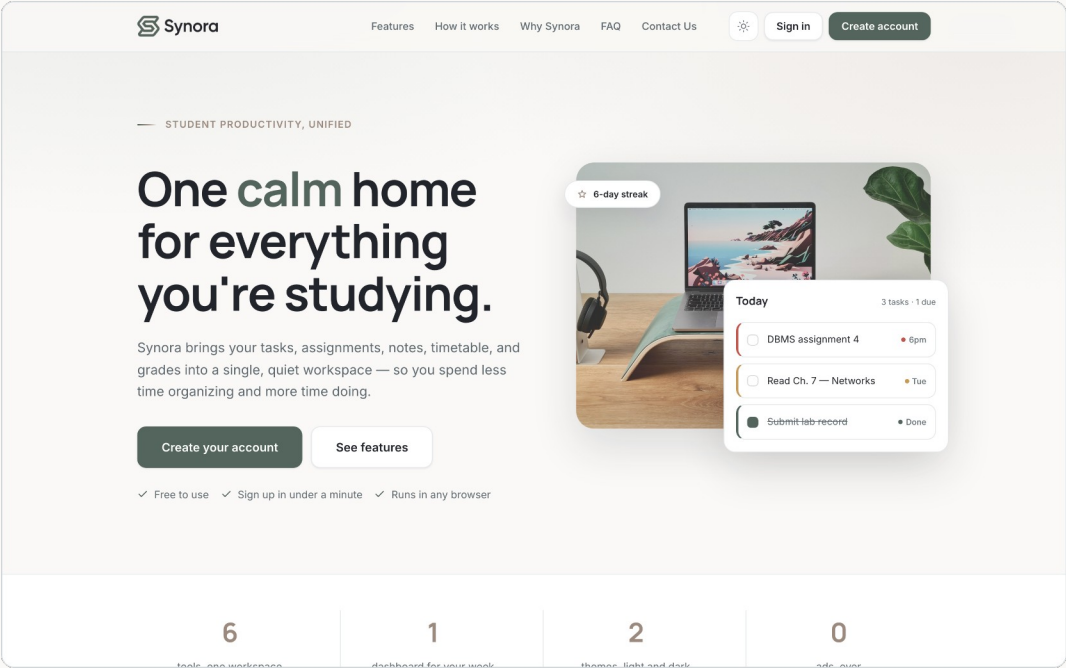
Flexbox

CSS Grid

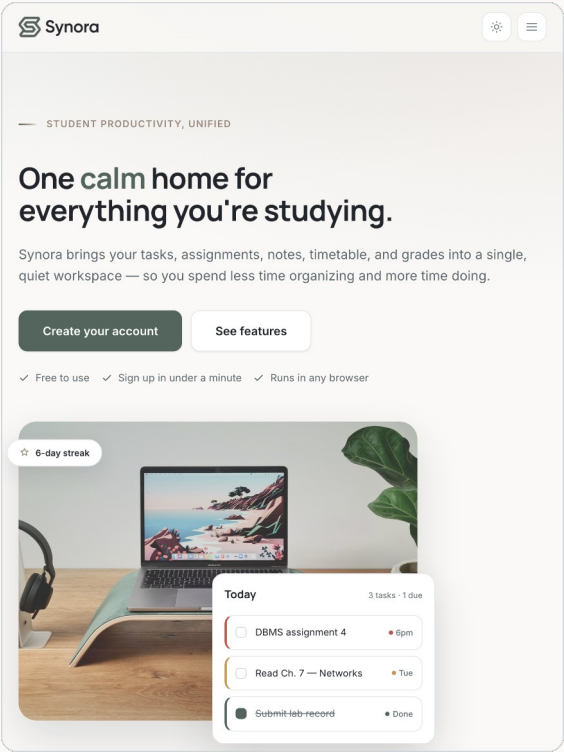
Media queries

One workspace. Every screen.

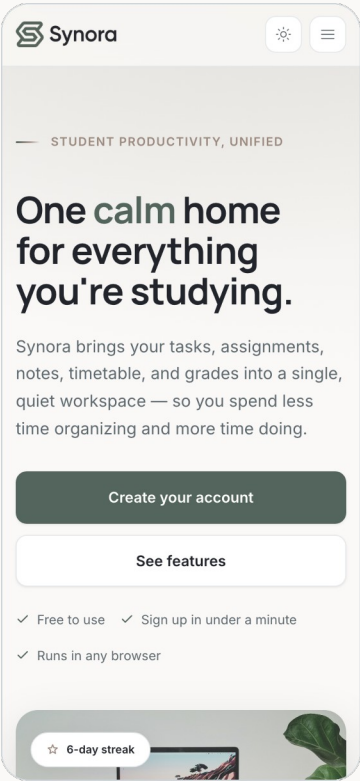
The same landing page at three real viewports. The full navbar collapses into a CSS-only hamburger menu, the two-column hero stacks into one, and every section reflows to a single column.



1440px • Desktop



768px • Tablet



390px • Mobile

Fluid units

Flexbox

CSS Grid

Media queries

Making the interface interactive

```
app.js - derive.taskStats

1 function taskStats() {
2   var tasks = store.get("tasks");
3   var stats = { total: tasks.length,
4     completed: 0, overdue: 0, dueToday: 0 };
5
6   tasks.forEach(function (t) {
7     if (t.status === "completed") {
8       stats.completed++; return;
9     }
10    var days = daysUntil(t.deadline);
11    if (days < 0)      stats.overdue++;
12    else if (days === 0) stats.dueToday++;
13  });
14  return stats;
15 }
```

1

Data

tasks in localStorage



2

JS logic

loops, objects, conditions



3

DOM

counts & lists built in code



4

UI

the dashboard updates

Fundamentals in one function: variables, an object, an array, a loop and conditionals — turning stored data into the numbers on the dashboard.

ES6+, used honestly

Synora is written in disciplined ES5-style syntax — **var**, **function**, **IIFE modules** — so it runs directly in the browser with no build step or transpiler. The modern features it uses are platform & library features, not syntax sugar.

✓ USED IN SYNORA

Promises — new Promise for confirm dialogs, consumed with .then()

ES6 array methods — find / findIndex to locate tasks by id

String.padStart — zero-padded local dates (ES2017)

Web Platform APIs — CustomEvent, matchMedia, localStorage, classList, dataset, Intl

— DELIBERATELY NOT USED

- Arrow functions, let / const
- Destructuring, spread / rest
- Template literals
- ES modules (import / export) — no bundler
- async / await, fetch — no backend to call

A good viva point: *the constraints are a deliberate choice, not a gap — the project targets the fundamentals cleanly.*

Event-driven UI

tasks.js

```
1 function toggleComplete(id) {  
2   var t = all().find(function (x) {  
3     return x.id === id; });  
4   if (t.status === "completed") {  
5     t.status = "not-started";  
6   } else {  
7     t.status = "completed";  
8     Synora.streak.recordCompletion();  
9   }  
10  store.set("tasks", list); // persist  
11  Synora.refresh();         // re-render  
12 }
```

Event delegation: one listener on the surviving container serves every task row — so re-rendering the list never loses its handlers.



Click a task's check button



Listener delegated on the list



Function toggleComplete(id)



Data status + streak saved



Re-render list & dashboard update



Feedback a toast confirms it

JSON • Storage • Forms

storage.js

```

1 function writeJSON(key, value) {
2   localStorage.setItem(key,
3     JSON.stringify(value));
4 }
5 function readJSON(key, fallback) {
6   var raw = localStorage.getItem(key);
7   if (raw === null) return fallback;
8   return JSON.parse(raw);
9 }
10 // synora:<user>:<collection>
11 function key(c) { return "synora:" + user + ":" + c; }

```

Forms sign-up • sign-in • personalization • subjects • tasks • contact — all validated inline in JavaScript, no browser bubbles.

JS object

\

JSON.stringify()

\

localStorage

\

JSON.parse()

\

JS object

\

UI

Honest limitation Browser storage gives persistence, but it is not a secure backend — anyone with the browser can read it. Passwords are hashed as a courtesy, not as security.

Beyond the syllabus



Attendance engine

$$\% = \text{attended} / \text{held} \times 100$$

Plus the real question: how many classes can I miss — or must I attend — to hit my target? The divide is guarded so 0 classes never shows NaN.



CGPA engine

$$\Sigma(\text{credits} \times \text{grade point}) / \Sigma \text{credits}$$

A single 10-point scale (O–B, plus E1/E2/E3). A failure scores 0 points but still carries its credits — so a backlog visibly drags the average down.



Subjects as source of truth

`one list, referenced by id`

Attendance, CGPA and the timetable never store a subject name. Rename once and it changes everywhere; delete once and it cascades.

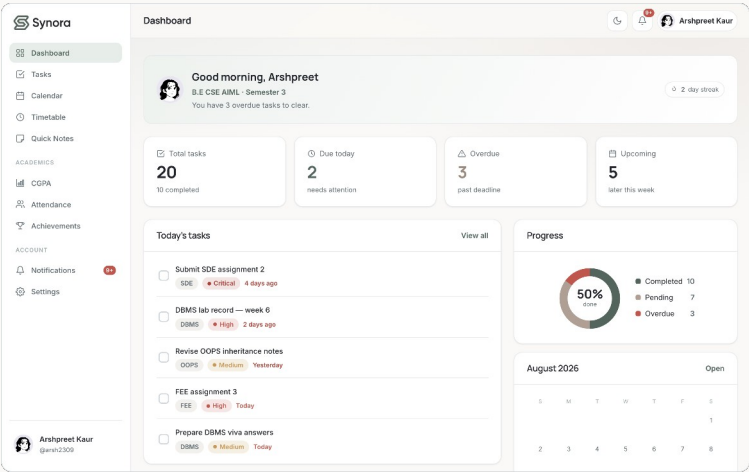


User-specific workspace

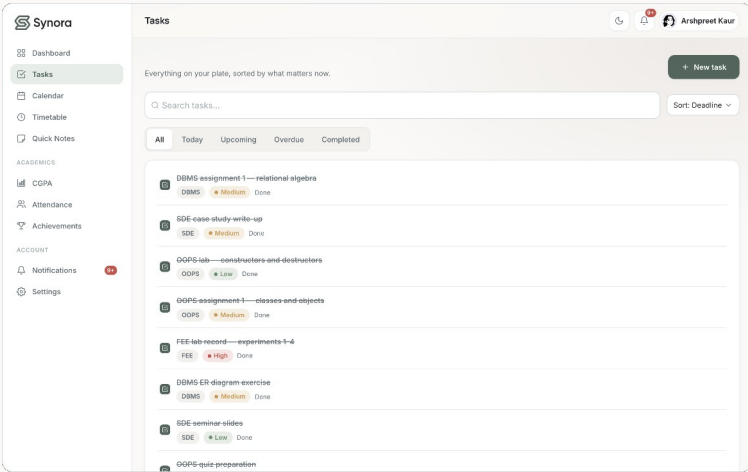
`synora:<user>:<collection>`

Each account is namespaced, so two students on one browser never see each other's data. Percentages, CGPA and streaks are derived, never stored.

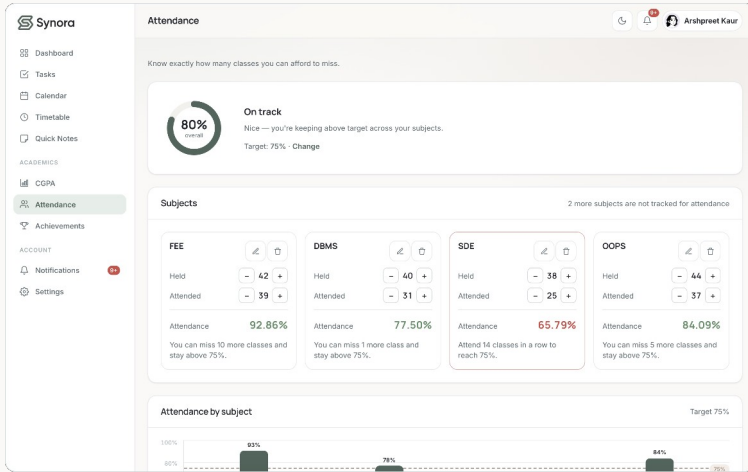
Synora in action



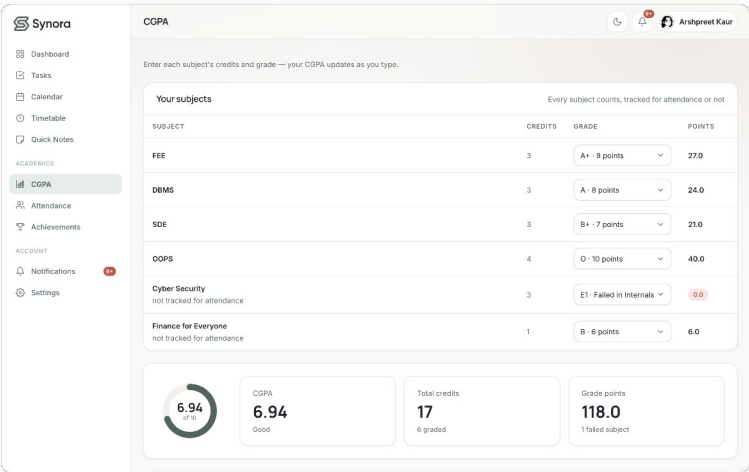
Dashboard



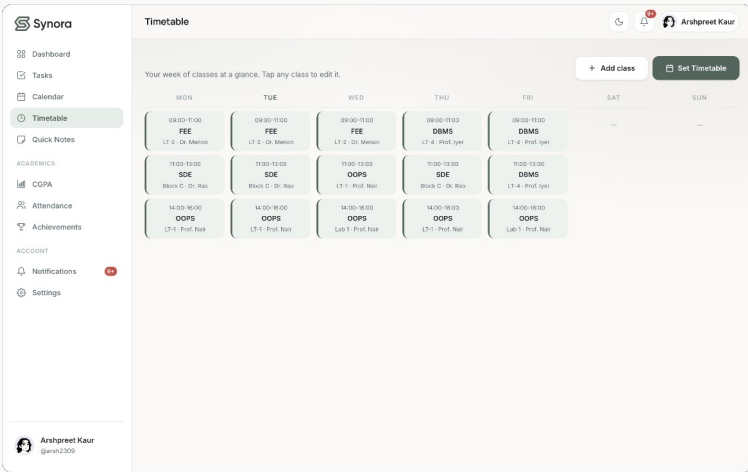
Tasks



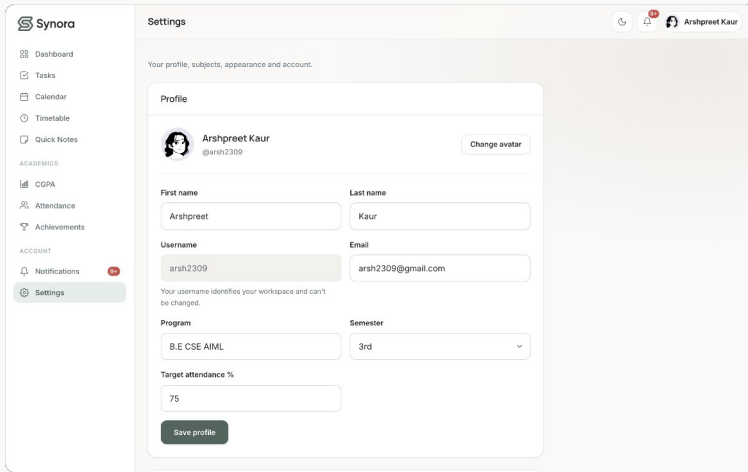
Attendance



CGPA

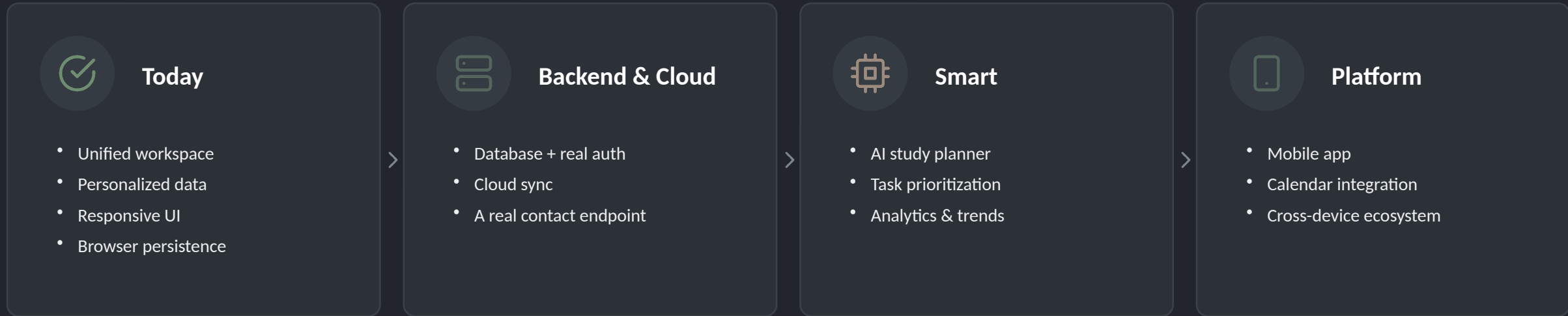


Timetable



Personalization

Where Synora goes next



Honest limitations Today's build is frontend-only — localStorage rather than a database, client-side auth, per-device, and a contact form that validates but doesn't send. A production version moves persistence & auth to a secure backend.



One calm home for everything you're studying.

Thank you • Questions?